CDT ML for Time Series



EHR Tokenization Practical

Organiser(s): ..... *Dr Patitapaban Palo*

Department of Engineering Science
University of Oxford

November 2026

Table of contents

1 Lab overview **1**

2 Data Loading and Understanding **2**
2.1 Data description . . . 2
2.2 Set up the notebook . . . 2
2.3 Task 1: Load and prepare data . . . 2

3 Understanding Tokenization **3**
3.1 Task 2: Understanding Triplet tokenization . . . 3
3.1.1 Instructions . . . 3
3.2 Task 3: Understanding EIC tokenization . . . 3
3.2.1 Instructions . . . 3
3.3 Task 4: Understanding Text-Code tokenization . . . 3
3.3.1 Instructions . . . 4
3.4 Task 5: Compare the representations . . . 4

4 Applying Tokenization to Mortality Prediction **5**
4.1 Task 6: Build the Input Embeddings . . . 5
4.1.1 Instructions . . . 5
4.2 Task 7: Implement the Transformer . . . 5
4.2.1 Instructions . . . 5
4.3 Task 8: Train and Compare Mortality Models . . . 6
4.3.1 Instructions . . . 6

5 Discussion **7**

1 Lab overview

The aim of this laboratory is to understand how clinical measurements can be turned into tokens for sequence models. We will use the same intensive care unit (ICU) dataset introduced in the earlier lab on EHR data preprocessing. The data are already prepared, so you do not need to repeat the preprocessing exercises.

You will first understand and explore three ways of representing clinical measurements: Triplet, Event-Interval-Code (EIC), and Text-Code tokenization. You will then use the tokens to predict mortality with a simple logistic regression model. This helps us see whether different representations of the same data lead to different results. The exercises are in the accompanying Jupyter notebook. Work through the notebook in order. You can use a local Jupyter environment or Colab. A GPU is not needed.

2 Data Loading and Understanding

2.1 Data description

The dataset contains 9,622 patient examples, with 48 time steps and 59 clinical features per example. Features include heart rate, oxygen saturation, respiratory rate, and temperature. The prediction target is a binary mortality label.

The provided data folder contains:

- `X_data.npy`: clinical measurements, arranged as patients × time steps × features.
- `y_data.npy`: mortality labels (0 = survived, 1 = died).
- `feature_names.txt`: the names of the 59 clinical features.

2.2 Set up the notebook

Place the extracted `data/` folder next to the notebook. If you use Colab, upload the folder or update the path to a copy in Google Drive. Run the setup and data-loading cells. These cells are provided in full; you do not need to edit them.

2.3 Task 1: Load and prepare data

The data loading and preparation steps are already provided in the Jupyter notebook. Run these cells before starting the tokenization exercises.

3 Understanding Tokenization

A token is a small unit of information that a model can process. A patient's history contains several kinds of information: what was measured, when it was measured, and its value. Tokenization is the step where we choose how to represent these parts.

For this section, we use four clinical features and the first 12 time steps of one patient. This keeps the examples short enough to inspect. The `LOW`, `MID`, and `HIGH` categories used in the exercises are based on data values, not clinical diagnostic cut-offs.

3.1 Task 2: Understanding Triplet tokenization

A triplet contains a time step, a feature name, and a value category. For example, `(2, Heart Rate, HIGH)` tells us which feature was measured, when it was measured, and its category.

3.1.1 Instructions

1. Complete the triplet function in the notebook.
2. Create one triplet for each selected measurement at each time step.
3. Print a few triplets and check how many are produced.

Refer to Task 1 in the notebook for the code and checks.

3.2 Task 3: Understanding EIC tokenization

EIC stands for Event-Interval-Code. This approach uses tokens to represent events and the gaps between them. In our exercise, we record a value category when it first appears or changes, together with the number of time steps since the previous recorded change.

3.2.1 Instructions

1. Complete the EIC function in the notebook.
2. Add a gap token, such as `GAP_3`, when three time steps pass between recorded changes.
3. Compare the number of tokens with Task 1.

Our ICU data are arranged on a regular time grid.

3.3 Task 4: Understanding Text-Code tokenization

Another approach is to turn measurements into text codes. A token might include a feature name and its category, such as `HeartRate_HIGH`. We can also include the time step if we want to keep temporal position explicit.

3.3.1 Instructions

1. Complete the Text-Code function in the notebook.
2. Print the first few tokens and compare them with the triplets.
3. Check how rounding or grouping numerical values changes the tokens.

3.4 Task 5: Compare the representations

Apply all three strategies to the same patient history. Compare their sequence lengths, number of distinct tokens, and the information they keep. Where the notebook asks you to reconstruct a value from a token, compare the recovered value with the original measurement.

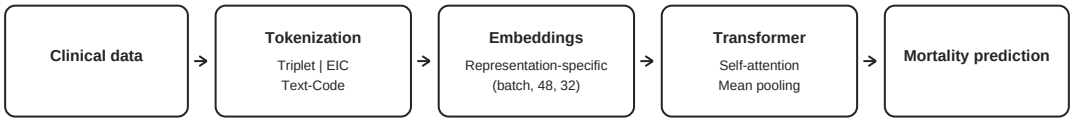


Figure 1: The tokenization strategy changes, while the downstream transformer remains the same.

4 Applying Tokenization to Mortality Prediction

In this section, you will use all 59 clinical features and all 48 time steps for each patient. Our goal is to predict mortality using a Transformer model.

We will compare the three tokenization strategies: Triplet, EIC, and Text-Code. Each strategy represents clinical information differently, so we need to convert its output into embeddings before passing it to the Transformer.

We use the same training, validation, and test patients for all three strategies to make the comparison consistent. The notebook provides the patient splits and prepares the value categories using training data.

4.1 Task 6: Build the Input Embeddings

A Transformer works with numerical vectors called embeddings. These vectors allow the model to learn patterns from a sequence of clinical measurements. Each tokenization strategy requires a different input representation. Although the input representations differ, they all should produce embeddings of the same size. This allows us to use the same Transformer architecture for all three strategies. Complete the embedding classes provided in the notebook.

4.1.1 Instructions

1. For Triplet, combine feature embeddings with feature-specific numerical values and time embeddings.
2. For EIC and Text-Code, create unique token IDs from the feature and value categories.
3. Convert the token IDs into embeddings and combine them with time embeddings.
4. Check that each input layer returns an output of shape `(batch, 48, 32)`.

4.2 Task 7: Implement the Transformer

We will use a Transformer encoder to learn patterns across the 48 time steps in each patient's history. The Transformer will take the input embeddings, process the sequence using self-attention, and produce a representation of the patient's history. We then use this representation to predict mortality. The model will use one Transformer encoder layer with two attention heads and an embedding dimension of 32. Complete the PatientTransformer class in the notebook.

4.2.1 Instructions

1. Pass the input data through the appropriate embedding layer.
2. Process the embeddings using the Transformer encoder.
3. Average the Transformer outputs across the time dimension.
4. Apply LayerNorm and the classification layer to produce one prediction logit.

4.3 Task 8: Train and Compare Mortality Models

We have now prepared the embeddings and built our Transformer model. The next step is to train the model to predict patient mortality. Code is provided for loading the data, setting up the optimizer and loss function, and evaluating the model.

4.3.1 Instructions

Complete the training loop in the `train_model()` function.

1. Reset the gradients and pass the patient sequences through the Transformer.
2. Calculate the prediction loss.
3. Use backpropagation to compute the gradients.
4. Evaluate the model on the validation set.
5. Save the best model weights when the validation AUROC improves.

5 Discussion

1. Which tokenization gave the highest AUROC? Was it also highest on AUPRC?
2. How might the different embedding strategies affect model performance?
3. Which strategy produces more model features? Is that always better?
4. Are we comparing only tokenization, or also the embeddings?

The goal is to understand how representation choices affect a simple downstream model.